

나의 목표가 가능하다면 전체 전류,전압 데이터를 이용해 그안의
전자기기 각각의 소비전력을 알아내는 거야

전체 전압과 전류 데이터를 활용해 개별 전자기기의 소비전력을 알아내는 것은 비침입형 전력 소모
관리(NILM, Non-Intrusive Load Monitoring)의 핵심 목표입니다.

전압 측정기를 추가하여 **전압(V)과 전류(I)를 함께 수집**할 수 있게 된다면, 기존의 전류 고조파
분석보다 훨씬 정밀하고 확실한 부하 분리(Disaggregation)가 가능해집니다.

1. 전압 데이터가 추가될 때 얻을 수 있는 결정적 강점

1. 유효전력(P) 및 무효전력(Q) 계산 가능

1. 전압과 전류의 위상차(θ)를 계산할 수 있어 $P = V \cdot I \cdot \cos \theta, Q = V \cdot I \cdot \sin \theta$ 를 실시간 산출할 수 있습니다.
2. $P - Q$ 평면 분석 : 기기마다 고유한 (P, Q) 좌표 범위를 가지므로, 기기 켜짐/
꺼짐 시 $P - Q$ 평면상의 이동 벡터를 통해 어떤 기기가 동작했는지 직관적으로
식별할 수 있습니다.

2. V-I trajectory (전압-전류 궤적) 핑거프린트 활용

1. 한 주기(16.67ms) 동안 전압 $v(t)$ 대비 전류 $i(t)$ 를 $X - Y$ 축에 그래프로 그리면
기기 고유의 폐곡선 형태(Trajectory)가 나타납니다.
2. 저항성 부하(헤어드라이어, 다리미) : 직선 형태
3. SMPS/용량성 부하(노트북, PC, TV) : 8자 모양이나 기울어진 원 형태
4. 유도성 부하(모터, 냉장고, 세탁기) : 넓은 타원 형태
5. 이 V-I 궤적 이미지를 CNN(합성곱 신경망)에 입력하면 높은 정확도로 기기를
분류해냅니다.

2. 목표 달성을 위한 전체 시스템 구현 단계

Step 1: 데이터 수집 및 전처리 (Data Acquisition)

- **고속 동기 샘플링** : 전압과 전류 신호를 동일한 시점에 동기화하여 수집합니다. (현재 수집
주기가 60Hz 기준 cycle 단위이므로 신호 동기화가 잘 유지되고 있습니다.)
- **기본 특징량 추출** : 각 Cycle/Frame 단위로 아래 항목들을 실시간 계산하여 저장합니다.
 - V_{rms}, I_{rms} , 유효전력 P , 무효전력 Q , 피상전력 S , 전력인수 PF
 - 전압/전류 차수별 고조파 크기 및 위상 ($IH_1 \sim IH_{15}, ihdeg_1 \sim ihdeg_{15}$)

Step 2: 단일 부하 핑거프린트 DB 구축 (Appliance Profiling)

개별 기기를 분리하기 전에 주요 가전기기들의 단독 동작 데이터베이스(DB)를 구축해야 합니다.

- **노트북 충전기** : 저전력, 높은 THD_I , 3/5/7차 고조파 우세, $PF \approx 0.5 \sim 0.7$
- **모니터/TV** : 중전력, SMPS 특성, 고조파 우세
- **전기포트/헤어드라이어** : 고전력(1000W+), THD_I 매우 낮음, $PF \approx 1.0$ (순수 저항)
- **냉장고/에어컨** : 모터 기동 시 높은 인러시 전류 발생, 무효전력 Q 비중이 큼

Step 3: 부하 분리 알고리즘 적용 (Disaggregation Engine)

목표로 하는 실시간성 및 시스템 복잡도에 따라 두 가지 방식 중 선택하거나 조합할 수 있습니다.

방식 A. 이벤트 기반 최적화 기법 (Event-based Optimization)

1. $P(t)$ 와 $Q(t)$ 의 급격한 변화(Event Step Change) 감지 $\Delta P, \Delta Q$
2. 이벤트 발생 순간의 전압-전류 궤적(V-I Trajectory) 및 고조파 변동량 계산
3. DB에 저장된 기기별 특성과 비교하여 켜진/꺼진 기기 매칭
4. **소비전력 산출** : 감지된 상태 기반으로 개별 기기의 $P_k(t)$ 분할

방식 B. 딥러닝 기반 파형 분리 (Sequence-to-Point 1D-CNN / UNet)

1. 전압/전류의 시계열 파형 $v(t), i(t)$ 또는 $P(t), Q(t)$ 데이터를 입력
2. 미리 학습된 Seq2Point Neural Network 모델 적용
3. 모델이 전체 전력 그래프에서 '노트북 충전기 소비전력', '모니터 소비전력' 파형을 각각 인퍼런스(Inference)하여 분리 출력

3. 성공적인 구현을 위한 실무 팁

- **전압 측정기 설치 필수** : 전압 변화에 따라 전류도 함께 변하므로, 정밀한 유효전력(W)과 위상차 계산을 위해서는 전압 수집이 반드시 필요합니다.
- **복합 부하 처리** : 여러 기기가 동시에 켜져 있을 때는 선형 중첩 원리($P_{total} = P_1 + P_2 + \dots$)에 따라 $P - Q$ 상태 공간 추적이나 NMF(비음수 행렬 분해) 기법을 함께 적용하면 정확도가 대폭 향상됩니다.
- **기존 수집 프레임 활용** : 현재 수집 중인 데이터 포맷(주기별 $I_{rms}, IH_1 \sim IH_{15}, ihdeg_1 \sim ihdeg_{15}$)은 NILM 특징 추출에 필요한 핵심 요소들을 이미 대부분 포함하고 있어 전압 데이터만 추가되면 곧바로 NILM 모델 개발로 확장 가능합니다.

방식 B. 딥러닝 기반 파형 분리 (Sequence-to-Point 1D-CNN / UNet)

RTX 5070 GPU(VRAM 12GB급 예상)와 노트북 구동 환경은 **Sequence-to-Point(Seq2Point) 기반 1D-CNN / UNet 모델**을 개발하고 실증하기에 매우 최적의 조합입니다.

RTX 5070은 수백만 개 이상의 파라미터를 가진 시계열 딥러닝 모델도 수십 분~수 시간 내에 여유롭게 학습시킬 수 있으며, 경량화된 모델은 인퍼런스(추론) 연산량이 적어 일반 노트북 CPU나 내장 GPU에서도 실시간(수십 ms 단위)으로 원활하게 구동됩니다.

이 방식의 구체적인 작동 원리부터 데이터 처리, 모델 구조, 그리고 학습/배포 전략까지 자세히 설명해 드립니다.

1. Sequence-to-Point (Seq2Point)의 핵심 개념

기존의 **Sequence-to-Sequence (Seq2Seq)** 방식은 "입력 파형 100개 지점 → 출력 파형 100개 지점"을 바로 예측하는 방식이라 입력과 출력 간의 시계열 노이즈나 위치 어긋남(Shift)에 취약했습니다.

반면 **Seq2Point** 방식은 다음과 같이 작동합니다.

1. **슬라이딩 윈도우(Sliding Window)**: 전체 전력/전류 시계열 데이터에서 목표 시점 t 를 중심으로 앞뒤 일정 길이(예: 599개 샘플, 약 10초 분량)의 입력 창(Input Window)을 잘라냅니다.
2. **단일 점 예측**: 딥러닝 모델은 이 10초 동안의 복합 파형 맥락(Context)을 분석하여, "**창의 맨 가운데 시점 t 에서 '노트북 충전기'가 소비하고 있는 전력값 $p_{laptop}(t)$** " 딱 하나(Single Point)만 예측합니다.
3. 슬라이딩 윈도우를 한 칸씩 우측으로 옮겨가며 전체 구간의 개별 기기 전력 곡선을 재구성합니다.

2. 입/출력 데이터 구성 (Data Pipeline)

(1) 입력 데이터 (Input Features)

수집된 데이터에서 각 시점 t 마다 다채널(Multi-channel) 1D 시계열 벡터를 생성합니다.

- **입력 텐서 크기**: (Batch_Size, Channels, Window_Length)
- **추천 채널 구성 (Channels)**:
 - $P(t)$ (유효전력) 및 $Q(t)$ (무효전력)
 - $I_{rms}(t)$ (실효 전류)

- 주요 고조파 성분: IH_1, IH_3, IH_5, IH_7 크기 및 위상차 $ihdeg_1, ihdeg_3 \dots$
(파형 샘플링 레벨 $v(t), i(t)$ 를 직접 넣을 수도 있으나, 주파수/고조파 요약 특징을 입력에 포함하면 모델 학습이 훨씬 빨라지고 안정화됩니다.)

(2) 출력 데이터 (Ground Truth Label)

- 학습 단계에서는 "합계 전력"과 함께 "노트북 충전기 전용 측정기로 만 개별 전력 $y(t)$ " 데이터를 라벨(Label)로 사용합니다.
- 손실 함수(Loss Function): Mean Squared Error (MSE) 또는 Mean Absolute Error (MAE)

3. 추천 모델 아키텍처 (1D-CNN vs 1D-UNet)

① 1D-CNN (기본 Seq2Point 모델)

- 구조: 1D Convolution Layer + Batch Normalization + ReLU → Global Average Pooling 또는 Dense Layer → Output (1 value)
- 특징: 구조가 매우 단순하고 가벼워 구현이 쉬우며, 노트북 환경에서 CPU 연산만으로도 수 ms 만에 인퍼런스할 수 있습니다.

② 1D-UNet (고정밀 파형 분리)

- 구조: 입력 파형을 압축하며 특징을 추출하는 Encoder(하향 수축) + 원본 해상도로 복원하는 Decoder(상향 확장) + Skip Connection.
- 특징: 시간적 맥락과 미세한 파형 변화를 동시에 보존하므로, 충전기처럼 전류가 스위칭되거나 변동이 심한 부하를 분리할 때 1D-CNN보다 훨씬 정밀한 전력 곡선을 복원해냅니다.

4. RTX 5070 활용 학습 전략 (Training Strategy)

RTX 5070의 뛰어난 컴퓨팅 성능을 100% 활용하기 위한 파이프라인입니다.

1. 데이터 합성 augmentation (Data Augmentation)

1. 단일 기기 데이터(노트북, 모니터, 조명 등)를 각각 수집한 후, 소스 코드로 이들을 무작위로 조합/합산하여 수십만 개의 가상 복합 전력 시계열 데이터를 자동 생성합니다.
2. RTX 5070 환경이라면 수십~수백만 개 배치(Batch)도 순식간에 학습 가능합니다.

2. 혼합 정밀도 학습 (FP16 / BF16 Mixed Precision)

1. PyTorch 기준 `torch.cuda.amp.autocast()` 를 적용하여 학습 속도를 2~3배 향상시키고 메모리 사용량을 절반으로 줄입니다.

3. Multi-Appliance Architecture (다중 출력 확장)

1. 하나의 모델이 '노트북 충전기' 하나만 예측하는 것이 아니라, 마지막 Output Layer의 출력 개수를 N 개로 늘려 [노트북 전력, 모니터 전력, 조명 전력] 을 한번에 동시에 추론하도록 설계할 수 있습니다.

5. 노트북 환경을 위한 실시간 구동 및 경량화 (Deployment)

학습이 끝난 모델을 노트북 환경(데이터 수집 프로그램과 연동)에서 실시간으로 구동하기 위한 핵심 기술입니다.

1. ONNX (Open Neural Network Exchange) 변환

1. PyTorch/TensorFlow로 학습한 .pt 파일 모델을 ONNX 포맷으로 Export합니다.

Python

```
# PyTorch 모델을 ONNX로 변환 예시
torch.onnx.export(model, dummy_input, "nilm_seq2point.onnx", ...)
```

2. ONNX Runtime (C++ / Python) 추론 Engine 적용

1. 노트북 환경에서는 PyTorch 전체 라이브러리를 띄우지 않고, 매우 가벼운 onnxruntime 파이썬 패키지나 C++ API를 사용해 추론합니다.
2. CPU 리소스 사용률을 5% 이하로 낮추면서 1회 추론에 1~5ms 이내로 처리 가능합니다.

3. 버퍼 기반 스트리밍 처리 (Buffer Queue)

1. 수집기에서 초당 60회(60Hz cycle) 데이터가 들어올 때, 최근 N개 샘플을 링 버퍼(Ring Buffer)에 쌓아두고 매 1초마다 ONNX 추론을 수행하여 화면에 각 기기별 소비전력 그래프를 갱신합니다.

6. 로드맵 및 실행 단계 추천

- **1단계 (데이터 수집 및 DB화):** 전압 센서 추가 후, 노트북 충전기 단독/모니터 단독/합성 동작 데이터를 각각 1~2시간 이상 수집합니다.
- **2단계 (파이프라인 구축):** PyTorch를 이용해 입력 창 길이(예: 300~600 cycle) 단위로 데이터를 잘라주는 Dataset / DataLoader 클래스를 작성합니다.
- **3단계 (RTX 5070 학습):** 1D-CNN 기반 Seq2Point 모델을 구성하고 PyTorch+AMP로 빠른 학습 및 Hyperparameter 튜닝을 진행합니다.
- **4단계 (노트북 최적화):** ONNX 변환 후 onnxruntime 으로 노트북 환경에서 기존 데이터 수집 코드와 결합하여 real-time 인퍼런스 테스트를 완료합니다.

코드 표시 <>

결론부터 말씀드리면 **현재 수집 중인 데이터 포맷만으로도 Seq2Point 딥러닝 기반 부하 분리(NILM) 모델 학습 및 실시간 구동이 충분히 가능합니다.**

오히려 일반적인 전력계보다 훨씬 풍부한 고급 특징량(High-level Features)을 이미 수집하고 있어 모델 성능을 높이기에도 매개변수 환경이 유리합니다.

현재 데이터 구조의 강점 분석

1. 높은 해상도 (60Hz Cycle 단위 샘플링)

- 초당 약 60번(16.67ms 마다)의 데이터가 들어오는 고속 시계열 데이터입니다.
- 기기 켜짐/꺼짐 순간의 과도 현상(Transient)과 미세한 전류 변화 패턴을 고해상도로 포착할 수 있습니다.

2. 풍부한 고조파 특징량 (전류 고조파 크기 및 위상)

- `ih1` ~ `ih15` (1~15차 전류 고조파 크기)
- `ihdeg1` ~ `ihdeg15` (1~15차 전류 고조파 위상각)
- 비선형 부하(노트북 충전기, TV, 모니터 등)는 차수별 고조파 크기와 위상의 조합이 기기마다 고유한 지문(Fingerprint) 역할을 합니다. Raw Waveform(생 파형)을 직접 넣지 않아도 15차 고조파 벡터 데이터만으로 개별 기기의 고유 파형 특성이 완벽히 설명됩니다.

3. 전압 데이터 추가 시 확보되는 추가 특징

- 전압 측정이 추가되면 `vrms`, `p_w` (유효전력), `phase_deg` (위상차) 및 `vh1` ~ `vh15` 데이터가 정상화되어 **유효전력(P)**, **무효전력(Q)**, **전력인수(PF)** 가 채널 벡터로 합쳐집니다.

입력 데이터 텐서 구성 예시

`Seq2Point` 모델에 입력할 때 1개 샘플(시점 t)당 아래와 같은 33개 채널(Channels)의 시계열 행렬로 재구성하여 사용하시면 됩니다.

- 기본 전력 데이터 (3채널):** `p_w`, `irms`, `phase_deg` (또는 `vrms`)
- 전류 고조파 크기 (15채널):** `ih1` ~ `ih15`
- 전류 고조파 위상 (15채널):** `ihdeg1` ~ `ihdeg15`

입력 창 (Input Window) 형태 예시:

- 윈도우 길이 : 300 cycle (약 5초 분량)
- 모델 입력 shape : (Batch_Size, 33_Channels, 300_Window_Length)
- 모델 출력 shape : (Batch_Size, 1) → Center point(150번째 시점)에서의 특정 기기 소비전력(W)

데이터 준비 시 체크해야 할 유일한 점

현재 수집 구조로 완벽히 가능하지만, 딥러닝 모델 학습을 위해서는 딱 한 가지만 추가해 주시면 됩니다.

- **지도학습(Supervised Learning)용 라벨 데이터 확보**
 - 학습할 때 입력은 '합성 전류/전력 데이터'이고, 정답(Ground Truth)은 '노트북 충전기 전용 플러그에서 개별 수집한 전력 데이터'입니다.
 - 따라서 데이터 수집 시 [합성 데이터 수집기]와 함께 [노트북 충전기 단독 측정기]를 동기화하여 동시에 수집하는 실험 데이터 세트 구축 과정만 거치시면 됩니다.

동기화가 필수적이야?

네, 딥러닝(Supervised Learning) 기반 NILM 모델을 학습시키는 단계에서는 **학습용 라벨(Label) 데이터와 입력 데이터 간의 시간 동기화가 사실상 필수적입니다.**

그러나 **실제 서비스 구동 단계에서는 동기화가 전혀 필요 없으며**, 학습 단계에서도 약간의 시간 오차를 극복하는 기술적 방법들이 존재합니다.

1. 왜 학습 단계에서 동기화가 필수적일까?

딥러닝 모델은 "합성 데이터의 특정 시점 패턴"과 "**정답(노트북 전력)의 동일한 시점 값**" 간의 관계를 1:1로 매칭하며 학습(Loss 계산)합니다.

- **시간 축이 어긋날 경우 (Time Shift) :**
 노트북을 꽂아 전력이 급증한 순간(Event)이 합성 데이터에는 $t = 10.0$ 초에 기록되었는데, 라벨 데이터에는 $t = 10.5$ 초에 기록되어 있다면 모델은 "**전류 스파이크가 튀었는데 왜 전력은 0W지?**" 하고 잘못된 패턴을 학습하게 됩니다.
- **학습 난조 및 성능 저하 :** 초당 60회(16.67ms)의 고속 수집 환경에서는 단 1초(60 cycle)의 시차만 발생해도 모델의 손실 함수(Loss)가 줄어들지 않고 학습이 수렴하지 못합니다.

2. 학습용 데이터 동기화를 맞추는 3가지 실무 방법

독립된 두 개 이상의 수집 장치(노트북 충전기 측정기 + 메인 전력 측정기)를 쓸 때 동기화를 확보하는 방법입니다.

1. 소프트웨어적 이벤트 타임스탬프 맞춤 (추천)

1. 양쪽 측정 프로그램에 동일한 시간(PC의 NTP 서버 시계 또는 `time.time_ns()`)을 기록합니다.
2. 실험 시작 시 "노트북 충전기를 꽂는 순간" 발생하는 명확한 전류 스파이크 지점을 두 데이터셋에서 찾은 뒤, 이 이벤트 시점을 $t = 0$ 으로 맞추는 **Cross-Correlation(상호 상관) 정렬** 전처리를 거칩니다.

2. 하드웨어 단일 수집기 사용

1. 데이터 수집 장치 하나에서 메인 선로(합성)와 노트북 전용 선로(라벨)의 CT 센서 2개를 동시에 연결하여 한 프로그램으로 수집하면 동기화 문제가 근본적으로 해결됩니다.

3. 손실 함수(Loss Function) 개선 (Soft Alignment)

1. 하드웨어적 동기화가 완벽하지 않다면, 단순 MSE 대신 시계열의 시차 어긋남을 완화해 주는 **DTW(Dynamic Time Warping) Loss**나 **Soft-DTW**를 Loss 함수로 채택하여 약간의 시간 오차를 모델이 스스로 흡수하도록 만듭니다.

3. 결론 요약

- **학습(Training) 단계** : 정답 데이터(라벨)를 만들 때 **동기화는 필수**입니다. (단, 실험 시작 전 전류 스파이크 이벤트 하나로 파이썬 코드를 통해 수동/자동 정렬이 가능하므로 하드웨어가 복잡해질 필요는 없습니다.)
- **추론(Inference/노트북 구동) 단계** : 동기화가 전혀 필요 없습니다. 메인 전력 선로에서 들어오는 V, I 데이터만 모델에 넣으면 모델이 '노트북 소비전력'을 실시간 추론해냅니다.

그럼 컴퓨터처럼 소비전력 변화가 큰 경우는 1번 방법으로 어떻게 극복해?

컴퓨터(PC, 노트북)처럼 소비전력이 초단위로 크게 출렁이는 부하는 동기화 오차가 조금만 발생해도 MSE(Mean Squared Error) 손실 함수가 폭등하여 학습이 실패하기 쉽습니다.

이런 가변 부하 환경에서 "1번 방법 (소프트웨어적 이벤트 타임스탬프 맞춤 + 정렬)"을 성공적으로 적용하는 핵심 기술 전략은 다음과 같습니다.

1. 교차 상관 분석 (Cross-Correlation)을 통한 ms 단위 자동 정렬

사람이 눈으로 파형을 보고 시간을 맞추는 대신, 두 데이터 간의 '신호 유사도'를 수학적으로 계산하여 시차(Time Shift)를 자동으로 찾아 맞춰줍니다.

1. **원리:** 메인 수집 데이터의 $I_{rms,total}(t)$ 파형과 노트북 단독 수집 데이터의 $P_{laptop}(t)$ 파형에 대해 **Cross-Correlation(상호 상관) 연산**을 수행합니다.
2. **정렬:** 상호 상관값이 최대가 되는 지점의 시차 Δt (예: +0.42초)를 계산합니다.
3. **적용:** 정답 데이터(라벨)의 시간 축을 $-\Delta t$ 만큼 이동(Shift)시키고, Pandas의 `merge_asof()` 기능을 사용해 ms 단위로 가장 가까운 타임스탬프끼리 정확히 1:1 매칭합니다.

2. 인위적 '동기화 마커(Marker) 이벤트' 발생시키기

컴퓨터의 작업 전력 변동에만 의존하지 않고, 실험 시작과 끝에 아주 명확한 '시그널'을 사람이 직접 만들어 주는 방식입니다.

- **방법:** 수집 시작 직후, 컴퓨터에서 **CPU 100% 과부하 벤치마크 프로그램(예: Cinebench, Prime95)**을 **3초간 실행 후 종료**합니다.
- **효과:** 전력이 $15W \rightarrow 100W \rightarrow 15W$ 로 급격하게 변하는 수직 수열 형태의 'Square Wave(직사각형 파형)'가 생성됩니다.
- 이 뚜렷한 **에지(Edge)** 지점을 기준으로 두 데이터를 맞추면 1 cycle(16.67ms) 오차 범위 내로 완전 동기화할 수 있습니다.

3. 'Dynamic Time Warping (DTW) Loss'로 학습 시 오차 흡수

하드웨어 시계의 미세한 흐름(Clock Drift) 때문에 시간이 지날수록 초당 수 ms씩 시간 어긋남이 누적될 수 있습니다. 이를 학습 단계의 **Loss 함수**로 극복합니다.

- **기존 MSE Loss:** t 시점의 예측값과 t 시점의 정답만 비교함 (시간축 어긋나면 Loss 폭발)
- **Soft-DTW Loss:** t 시점 근처(± 5 cycle 내)의 파형 흐름 자체가 유사한지를 평가함
- **결과:** 컴퓨터처럼 전력이 가변하더라도, 파형의 '형태(Shape)'와 '변화 패턴'을 중심으로 학습하므로 시차 누적에 매우 강건(Robust)해집니다.

4. 컴퓨터 특화 데이터 전처리 (Smoothing & Augmentation)

- **Smoothing (이동 평균):** 아주 미세한 고주파 노이즈 변동에 모델이 과적합(Overfitting)되지 않도록 3~5 cycle 수준의 가우시안 필터나 이동 평균을 살짝 적용해 라벨을 가공합니다.
- **Data Augmentation:** 정렬된 학습 데이터에 일부러 $\pm 1 \sim 3$ cycle 정도의 시간 Shift를 준 합성 데이터를 추가 학습시켜, 모델이 약간의 시차 어긋남이 있어도 정상 추론하도록 유연성을 줍니다.

요약: 추천 데이터 가공 흐름

1. 수집 시작 직후 '고부하 벤치마크 3초 켜기'로 동기화 마커 생성
2. Python의 `scipy.signal.correlate` 로 두 데이터간 최적 Δt 도출 후 데이터 밀착 정렬
3. PyTorch 학습 시 **Soft-DTW Loss** 또는 **Smoothed L1 Loss** 채택

그럼 이런 방식이 가능해?

학습시킬 전자기기들을 단일 측정기로 각각 하나씩 같은 시간 동안 측정하고 그걸 합성 시켜서 하나의 측정기로만 합성과 정답을 모두 만드는 거지

네, 완벽하게 가능하며 실제로 학계와 산업계(NILM 분야)에서 가장 권장하는 최고의 데이터 생성 방식입니다.

이 방식을 **합성 데이터 생성 (Data Augmentation / Synthetic Data Generation)** 이라고 부릅니다. 질문하신 구조를 활용하면 동기화 문제와 데이터 수집의 한계를 한 번에 해결할 수 있습니다.

이 방식이 강력한 이유 (3가지 핵심 장점)

1. **100% 완벽한 시간 동기화 (Zero Time Shift)**
 1. 단일 측정기로 구한 기기 $A(t)$ 와 기기 $B(t)$ 의 개별 파형을 파이썬 코드로 단순히 더해서 $S(t) = A(t) + B(t)$ 를 만듭니다.
 2. 소스 코드 상에서 이미 완벽한 위치에 정답(Label)과 합성 입력(Input)이 매칭되므로, **ms 단위의 시차 오차가 근본적으로 0**이 됩니다.
2. **무한대에 가까운 학습 데이터 생성 (Data Augmentation)**
 1. 기기 3개(노트북, 모니터, 조명)를 각각 1시간씩만 측정해도, 합성 조합(노트북만 켜짐, 모니터+조명, 3개 모두 켜짐 등) 및 켜지는 시점(Time Offset)을 다르게 주면 **수백 시간 분량의 다양한 가상 복합 파형**을 합성해낼 수 있습니다.
 2. RTX 5070 환경에서 이 대량의 합성 데이터를 학습시키면 모델의 일반화 성능이 극대화됩니다.
3. **고조파 복합 파형의 선형 중첩성 활용**
 1. 전력망의 전류 및 고조파 성분은 복합 부하 상태에서 선형적 중첩(Additive)되는 성질이 강합니다.
 2. 즉, $I_{total}(t) \approx I_{laptop}(t) + I_{monitor}(t)$ 및 고조파 벡터의 합이 실제 멀티탭에 두 기기를 함께 꽂았을 때의 파형과 매우 유사합니다.

파이썬(Python)으로 구현하는 데이터 합성 로직 (예시)

실제 학습 데이터를 만들 때 파이썬 코드로 아래와 같이 구현합니다.

Python



```
import numpy as np
import pandas as pd

# 1. 단일 측정기로 수집한 개별 기기 데이터 로드
df_laptop = pd.read_csv("laptop_only.csv") # 노트북 충전기 단독 측정 데이터
df_monitor = pd.read_csv("monitor_only.csv") # 모니터 단독 측정 데이터
df_noise = pd.read_csv("background_noise.csv") # 대기 전력 / 노이즈 데이터

# 2. 파형 합성 (Input Feature 생성)
# 주요 특징량(Irms, 고조파 등)을 각각 더하거나, 복소 고조파 벡터를 합성
df_synthetic_input = pd.DataFrame()
df_synthetic_input['p_w'] = df_laptop['p_w'] + df_monitor['p_w'] + df_noise['p_w']
df_synthetic_input['irms'] = np.sqrt(df_laptop['irms']**2 + df_monitor['irms']**2) # RM

# 고조파 성분 합성 (1~15차)
for i in range(1, 16):
    df_synthetic_input[f'ih{i}'] = df_laptop[f'ih{i}'] + df_monitor[f'ih{i}']

# 3. 정답(Label) 데이터 설정
# Target Label A : 노트북 전력 (p_w)
label_laptop = df_laptop['p_w']
# Target Label B : 모니터 전력 (p_w)
label_monitor = df_monitor['p_w']
```

성공적인 모델 학습을 위한 2가지 실무 주의점

이 방식은 매우 훌륭하지만, 실제 환경과 완벽히 동일하게 만들기 위해 **합성 시 아래 2가지를** 추가해주셔야 합니다.

1. 배경 노이즈 및 대기전력(Background Noise) 추가

1. 집이나 사무실의 콘센트에는 기기를 켜지 않아도 미세한 고조파 노이즈나 기본 대기전력이 흐릅니다.
2. 아무것도 켜지 않은 상태의 **기본 선로 노이즈 데이터**를 10~20분간 수집한 뒤, 합성 파형에 무작위로 살짝 엮어주면 모델이 노이즈에 견고해집니다.

2. 최종 검증(Validation)은 실제 동시 작동 데이터로 수행

1. **학습(Training)** : 위에서 만든 '합성 데이터'로 RTX 5070에서 대량 학습 진행

2. **검증(Test/Validation)** : 실제로 멀티탭에 노트북과 모니터를 동시에 꽂고 측정한 **진짜 복합 데이터**를 준비하여, 합성 데이터로만 학습한 모델이 실제 환경에서도 잘 분리해내는지 최종 확인

LUT based로 지금 시뮬레이션 한다는거 같은데 근데 실제로는 노이즈도 있을거고 피상, 무효는 어케 구별할거임? 들어가는 전류가 교류아님? PF가 항상 일괄적이라는 보장이 있나?

질문하신 내용들은 LUT(Look-Up Table) 기반 NILM 시뮬레이션을 실제 현장 시스템으로 구현할 때 맞닥뜨리는 **가장 날카롭고 현실적인 한계점들**입니다.

언급하신 세 가지 핵심 이슈(노이즈, AC 상의 P/Q 구별, 가변 PF)를 실제로 어떻게 해결하는지 체계적으로 설명해 드립니다.

1. AC 환경에서 피상/유효/무효 전력 구별 및 계산

교류(AC) 환경에서도 동기화된 전압 $v(t)$ 와 전류 $i(t)$ 데이터만 있다면 P, Q, S는 수학적으로 명확하게 구별 및 계산됩니다.

- **실효값(RMS) 계산:**

$$V_{rms} = \sqrt{\frac{1}{T} \int_0^T v(t)^2 dt}, \quad I_{rms} = \sqrt{\frac{1}{T} \int_0^T i(t)^2 dt}$$

- **피상전력(S):** 단순 RMS의 곱입니다.

$$S = V_{rms} \times I_{rms} \quad [\text{VA}]$$

- **유효전력(P):** 한 주기 동안의 instantaneous power(순시 전력) 평균값입니다.

$$P = \frac{1}{T} \int_0^T v(t) \cdot i(t) dt \quad [\text{W}]$$

- **무효전력(Q):** 기본파(60Hz) 기준 전압과 전류의 위상차 θ (현재 데이터의 `phase_deg`)를 이용해 산출하거나, 고조파가 포함된 비선형 부하에서는 아래 관계식으로 완벽히 분리합니다.

$$Q = V_{rms} \cdot I_{rms,1} \cdot \sin(\theta_1) \quad [\text{var}]$$

$$S^2 = P^2 + Q^2 + D^2 \quad (D \text{는 고조파 왜곡 전력 Distortion Power})$$

현재 수집 중인 데이터 포맷에는 `irms`, `vrms`, `p_w`, `phase_deg` 및 차수별 위상(`ihdeg`)이 모두 포함되어 있으므로 AC 환경이라도 P , Q , S , D 가 실시간으로 완벽히 구별됩니다.

2. 역률(PF)이 일괄적이지 않고 변하는 문제 극복

노트북 충전기나 모니터 같은 SMPS 부하는 부하율(Load Level, 예: CPU 10% 사용 vs 100% 사용)에 따라 역률(PF)과 위상각이 지속적으로 변합니다.

LUT 방식의 한계 및 해결책

- 1차원 LUT의 한계:** 단순히 "노트북 = PF 0.6"처럼 고정값(단일 스칼라)으로 테이블을 만들면 부하가 변할 때 완전히 틀어집니다.
- 다차원 LUT (Multi-dimensional LUT) 구축:**
단순 기기명이 아니라 전류 크기(I_{rms})나 유효전력(P)의 구간별로 PF 및 고조파 프로필을 동적으로 매핑합니다.
 - 예: [노트북_P_15W -> PF 0.52, IH3 85%], [노트북_P_65W -> PF 0.88, IH3 42%]
- 딥러닝(Seq2Point/1D-CNN)의 우위:**
질문하신 이유 때문에 실제 환경에서는 LUT 방식보다 딥러닝 방식이 훨씬 선호됩니다. 딥러닝 모델은 입력으로 들어오는 P , Q , $IH_1 \sim IH_{15}$ 의 복합적 변화 패턴을 학습하므로, 역률이 실시간으로 변하더라도 연속적인 곡선(Non-linear mapping)으로 정확히 추적해냅니다.

3. 실제 환경의 노이즈(Noise) 처리 방안

실제 선로에는 전자기적 간섭(EMI), 다른 미세 기기의 대기전력, 전압 변동 등의 노이즈가 항상 존재합니다.

분류	노이즈 종류	대응 기법 및 극복 알고리즘
상시 노이즈	기본 선로 노이즈, 대기전력	배경 노이즈 감산 (Background Subtraction) 기기를 아무것도 켜지 않았을 때의 I_{rms} 및 고조파 바닥값(Floor level)을 측정하여 주기적으로 새도우 맵에서 차감

분류	노이즈 종류	대응 기법 및 극복 알고리즘
고주파 스파이크	스위칭 노이즈, 순간 서지	시계열 이동평균 및 가우시안 필터ing 초당 60회 샘플링되므로 3~5 cycle Window 단위로 이동 median/mean 필터를 적용해 튀는 스파이크 제거
미분류 부하	DB에 없는 새로운 기기	잔여 전력(Residual Power) 할당 $\text{Residual} = P_{total} - \sum P_{recognized}$ 형태로 남겨두고, 이 잔여량이 일정 기준 이하이면 노이즈로 처리
...		

요약 및 결론

- AC 전원에서의 구별:** 전압/전류의 실시간 곱(순시 전력)과 위상차(`phase_deg`)를 통해 유효전력(P)과 무효전력(Q)은 AC 상에서도 완벽히 분리됩니다.
- 가변 PF 문제:** LUT 방식을 쓰려면 '부하 수준별 동적 LUT'를 구축해야 하며, 이를 가장 자연스럽게 해결하는 것이 앞서 논의한 **딥러닝(Seq2Point) 방식**입니다.
- 노이즈 극복:** 대기전력 노이즈를 별도 채널로 측정하여 **합성 데이터 학습 시 노이즈를 미리 섞어주는 Data Augmentation** 기법을 적용하면 실제 현장에서도 높은 견고성(Robustness)을 확보할 수 있습니다.

DL이라고 해도 지금 PF의 변화량이 아마 non homgeous할건데 그러면 이러한 특유의 굴곡이나이러걸 몇개의 point에 대한 데이터를 기반으로 superivsed할 수 있나?

결론부터 말씀드리면 **지도학습(Supervised Learning)** 으로 충분히 극복 및 추론이 가능합니다.

딥러닝의 다층 레이어와 비선형 활성화 함수(ReLU, GELU 등)는 비동질적(Non-homogeneous)이고 불규칙하게 변하는 복잡한 함수 특성을 곡선 형태로 정밀하게 복원해내는 **보편적 함수 근사기(Universal Function Approximator)** 역할을 하기 때문입니다.

대표 포인트 데이터 기반으로 PF 굴곡을 극복하는 원리

- **고차원 특징 공간 상의 매끄러운 보간** : 단순 PF 스칼라값 하나만 관찰할 때는 굴곡이 불규칙해 보이지만, 모델은 $P, Q, IH_1 \sim IH_{15}, ihdeg_1 \sim ihdeg_{15}$ 가 만드는 30여 개 고차원 특징 공간 전체를 학습합니다. 고차원 공간으로 확장되면 1차원상의 비동질적 굴곡이 매끄러운 궤적(Trajectory)으로 펼쳐지므로, 몇 개의 대표 작동 지점만으로도 사이 구간을 정밀하게 보간(Interpolation)해냅니다.
- **비선형 활성화 함수의 곡선 피팅** : 부하율에 따른 PF 변화 특성은 신경망 가중치와 비선형 활성화 함수의 조합을 통해 비선형 곡선 형태로 피팅됩니다. Idle 상태, 중간 부하, 최대 부하 등 몇 가지 주요 작동 상태만 학습되어도 미학습 구간의 전력대 PF 변동 특성을 유연하게 추적합니다.

불연속 포인트 데이터의 오차를 최소화하는 3가지 실무 전략

- **동적 부하 스위핑(Dynamic Workload Sweeping) 수집** : 단속적인 고정 포인트(예: 15W, 30W, 60W) 데이터만 수집하기보다, 컴퓨터에서 벤치마크 프로그램을 실행하여 전력이 0%에서 100%까지 연속적으로 움직이는 2~3분 분량의 동적 파형을 수집합니다. 단 한 번의 수집으로 PF 의 전체 연속 굴곡 구간을 완벽히 포함할 수 있습니다.
- **복소 고조파 벡터 보간(Data Augmentation)** : 수집된 포인트 A와 포인트 B 사이의 비어 있는 전력 구간에 대해, 고조파 크기 IH 와 위상각 $ihdeg$ 를 복소 공간에서 연속적으로 이어서 가상의 동적 파형 데이터를 합성하여 학습에 추가합니다.
- **물리 제약 손실 함수(Physics-Informed Loss)** : $S^2 = P^2 + Q^2 + D^2$ 와 같은 교류 전력 물리 관계식을 Loss 함수에 패널티 항으로 포함시킵니다. 데이터가 부족한 중간 구간에서도 모델이 물리 법칙을 벗어나는 엉뚱한 PF 나 전력값을 출력하지 않도록 제약할 수 있습니다.

애초에 sequencial 하면 강 GRU나 attention을 쓰는게 더 좋지 않나? PINN을 사용해서 제약하려면 실측 데이터와의 노이즈같은 오차를 고려하기 힘들텐데

매우 깊이 있고 정확한 지적입니다. 말씀하신 두 가지 핵심 질문——① CNN/UNet 대신 GRU나 Transformer(Attention)를 쓰는 것이 맞지 않나? 그리고 ② PINN 적용 시 노이즈로 인한 물리식 패널티의 한계——에 대해 NILM 실무 관점에서 명쾌하게 비교 및 해답을 정리해 드립니다.

1. Sequential한 시계열인데 왜 GRU/Attention 대신 1D-CNN/UNet을 쓸까?

이론적으로 시계열 흐름은 GRU, LSTM, Transformer(Attention)가 가장 자연스러워 보이지만, NILM 분야에서 Seq2Point 구조 구현 시 1D-CNN / UNet이 표준(SOTA)으로 자리 잡은 실용적인 이유가 있습니다.

비교 항목	1D-CNN / 1D-UNet	GRU / LSTM	Attention / Tra
특징 추출 대상	파형의 주기적 패턴/ 고조파 형태(Local Pattern)	시간에 따른 상태 변화 및 전이(State Transition)	먼 과거/미래와 연관성(Long Dependency)
연산 속도 / 병렬화	극도로 빠름 (완벽한 병렬 연산, GPU 최적화)	느림 (이전 시점 출력 h_{t-1} 이 있어야 연산 가능)	메모리 소모 큼 $O(N^2)$ Self- 비용)
노트북 구동 (ONNX)	1~2ms 이내 (CPU 부담 거의 없음)	10~30ms (순차 연산 병목 발생)	수십 ms ~ 수백

...

핵심 원인: NILM의 Seq2Point는 '장기 기억'보다 '단기 파형 특징'이 중요함

- 노트북 충전기의 전류 파형 분리는 **1초 전, 10초 전의 상태가 어땠는지(Long-term dependency)보다**, 현재 타임 윈도우(약 3~5초, 200~300 cycle) 안의 **고조파 비율, 위상 왜곡, 펄스 형태(Local Feature)를 포착하는 것**이 훨씬 결정적입니다.
- CNN의 Filter(Kernel)가 시계열 축을 훑으면서 이 고조파 패턴을 아주 정밀하게 잡아냅니다.

💡 실무 결론 & 추천 구조

만약 순차적 맥락(Sequential Context)을 꼭 반영하고 싶다면, 1D-CNN + GRU 혼합 구조(CRNN)를 사용합니다.

- **1D-CNN Encoder**로 고조파/파형 특징을 먼저 압축 추출한 뒤, 그 뒤에 가벼운 GRU Layer(1~2층)를 붙여 시간적 흐름을 보정하는 방식이 단독 GRU나 Transformer보다 성능 및 속도 면에서 훨씬 뛰어납니다.

2. PINN(Physics-Informed Neural Network)과 실제 노이즈 간의 충돌 문제

"실측 데이터의 노이즈 때문에 물리 방정식($S^2 = P^2 + Q^2 + D^2$ 등)을 등식(Equality)으로 강제하면 오버피팅이나 학습 방해가 일어난다"는 지적 역시 완전히 타당합니다.

실제 현장 선로에는 고주파 EMI 노이즈, 측정 센서의 1~2% 계측 오차, 미세한 기선(Baseline) 변동이 존재하므로 엄격한 물리 방정식을 강제하면 모델이 수렴하지 못합니다.

실제 현장(Real-world)에서의 해결책: Soft Physics Constraint (완화된 제약)

PINN을 등식 하드 제약(Hard Constraint)이 아닌, **노이즈 마진(Margin)**을 허용하는 등구간 손실 함수 형태로 변형하여 적용합니다.

1. 상한/하한 불등식 패널티 (Boundary Penalty)

1. 물리식 $P_{laptop} \leq P_{total}$ 을 엄격히 따르되, 노이즈 오차 범위를 감안하여 약간의 마진 ϵ 을 둡니다.
2. 예: 예측된 개별 전력의 합이 전체 측정 전력 $+ \epsilon$ 을 초과할 때만 패널티 부여

$$\mathcal{L}_{physics} = \max \left(0, \sum P_{predicted} - (P_{total} + \epsilon) \right)$$

2. 물리 Loss의 가중치(Weight) 스케일링

1. 전체 손실 함수에서 데이터 기반 Loss(MSE)와 물리 Loss의 비중을 조절합니다.

$$\mathcal{L}_{total} = \mathcal{L}_{MSE} + \lambda \cdot \mathcal{L}_{physics} \quad (\lambda \approx 0.01 \sim 0.05)$$

2. 물리식은 모델이 엉뚱한 값(예: 음수 전력 출력, 전체 전력을 뛰어넘는 출력)으로 튀는 것을 막아주는 **가이드레일(Safety Rail)** 역할만 수행하게 합니다.

요약 정리

1. **모델 선택:** NILM 특성상 Local Pattern(고조파/파형) 추출이 핵심이므로 **1D-CNN / UNet**이 속도와 정확도 면에서 **최고의 선택**입니다. 순차성을 더하고 싶다면 **CNN + GRU 하이브리드**를 권장합니다.
2. **PINN & 노이즈:** 실측 노이즈를 고려해 완벽한 등식 제약이 아닌 마진을 둔 오차 범위 내 불등식 패널티(Soft Constraint)로 적용하면 노이즈에 견고하면서도 물리학적 정당성을 갖춘 모델을 만들 수 있습니다.

NILM에서 전압은 2Hz 전류는 60hz로 데이터가 들어와도 학습에 지장이 있을까?

결론부터 말씀드리면, 전압(2Hz)과 전류(60Hz)의 샘플링 주기가 달라도 NILM 모델 학습에 지장이 없으며, 실제로도 매우 흔하게 처리하는 방식입니다.

전압 신호는 계통 상에서 급격하게 변하지 않고 비교적 안정적(220V RMS 근처에서 완만하게 변함)인 반면, 전류 신호는 가전기기의 스위칭 및 부하 변동에 따라 순간적으로 크게 출렁이고 고조파 특성을 포함하기 때문입니다.

1. 두 주기가 다를 때 지장이 없는 이유

- **전압의 물리적 특성** : 일반적인 가정/사무실 전원 환경에서 전압의 RMS값(V_{rms})이나 기본 주파수(60Hz)는 수초 동안 큰 변화 없이 일정하게 유지됩니다. 따라서 2Hz(0.5초 간격) 정도의 전압 업데이트만으로도 전류의 전력(P, Q) 계산을 위한 전압 기준값을 충분히 제공할 수 있습니다.
- **정보의 밀도 차이** : NILM에서 기기를 식별하는 핵심 지문(Fingerprint) 데이터는 대부분 전류의 고조파($IH_1 \sim IH_{15}$), 전류 위상각, 전류 RMS(I_{rms})에 몰려 있습니다. 전류가 60Hz 고해상도로 유지된다면 기기 특성 추출에는 문제가 없습니다.

2. 학습 전 전처리 방법 (2가지 접근법)

딥러닝 모델(1D-CNN, Seq2Point 등)의 입력 텐서(Input Tensor) 형태를 맞추기 위해 파이썬(Pandas/NumPy) 전처리 단계에서 아래 두 가지 중 하나를 선택해 적용하시면 됩니다.

방법 A. 전압 데이터 업샘플링 (Up-sampling & Interpolation) — 가장 추천

전압 데이터(2Hz)를 보간(Interpolation)하여 전류 데이터(60Hz)와 동일한 타임스탬프 축으로 맞춰주는 방식입니다.

- **처리 과정:**
 1. 2Hz 전압 데이터 사이의 빈 시점(약 30개 cycle 분량)을 **선형 보간(Linear Interpolation)** 또는 최근접 보간(Forward Fill)으로 채웁니다.
 2. 전류(60Hz)와 전압(60Hz)이 완전 동기화된 60Hz 다채널 시계열 데이터셋을 생성합니다.
- **장점:** 기존의 60Hz 기반 1D-CNN / UNet 모델 구조를 수정 없이 그대로 사용할 수 있습니다.

방법 B. 특징량 별도 입력 (Multi-input Architecture)

전류 시계열 파형(60Hz)과 전압/전력 스칼라 정보(2Hz)를 딥러닝 모델의 서로 다른 레이어로 입력받는 방식입니다.

- **처리 과정:**
 - **Branch 1 (60Hz 전류 데이터):** 1D-CNN을 통과시켜 전류의 고조파/파형 특징 추출
 - **Branch 2 (2Hz 전압 데이터):** Dense Layer를 통과시켜 전압 수준 특징 추출
 - 두 Branch의 Feature Vector를 **Concatenate** 하여 최종 소비전력 추론

3. 유의해야 할 실무 체크포인트

1. 유효전력 $P(t)$ 산출 시 주의

1. 수집기 자체에서 $V(t)$ 와 $I(t)$ 의 순시 곱을 통해 매 cycle(60Hz)마다 p_w (유효전력)를 직접 계산해서 넘겨주는 구조라면, 전압 수집 주기가 2Hz라도 P 계산

자체는 60Hz 해상도로 이루어지므로 아무런 손실이 없습니다.

2. 만약 서버/PC에서 $V_{rms} \times I_{rms}$ 로 P 를 후계산하는 구조라면, 업샘플링(Interpolation)을 거친 후 곱해주시면 됩니다.

2. 전압 변동이 심한 환경 (Sag/Swell)

1. 공장 지대나 전력망이 불안정하여 0.5초(2Hz) 이내에 전압 강하(Sag)가 크게 발생하는 환경이라면 2Hz 전압 수집이 오차 요인이 될 수 있습니다.
2. 일반적인 상업/가정용 환경이라면 2Hz 전압 + 60Hz 전류 조합으로도 **95% 이상의 높은 부하 분리 정확도**를 얻을 수 있습니다.